

Stack ADT and Array-based Stacks

Algorithms and Data Structures
NEU 2026

Linear Structures

- Part 1

- Linear Structures
- Stack ADT and Array-based Stacks

- Part 2

- Queue ADT and Array-based Queues

- Part 3

- List and Iterator ADTs
- Amortisation

General Linear Structures

A linear structure is one whose elements can be seen as being in a line or sequence. That is, one element follows the next.

- Stacks
- Queues
- Lists
- Vectors
- Sequences

Implementing Linear Structures

□ Array

- Data is stored in consecutive array positions
- Operate on element at an index position

□ Linked Structure

- Elements are linked together to create linear structure
 - ◆ e.g. Linked List

Array Implementation

- ❑ Sequence of consecutive memory cells
- ❑ Size must be specified at creation
 - Generally can't increase size
 - ◆ memory immediately after the array may be allocated for other purposes
 - ◆ When supported, costly to extend
 - Generally can't decrease size

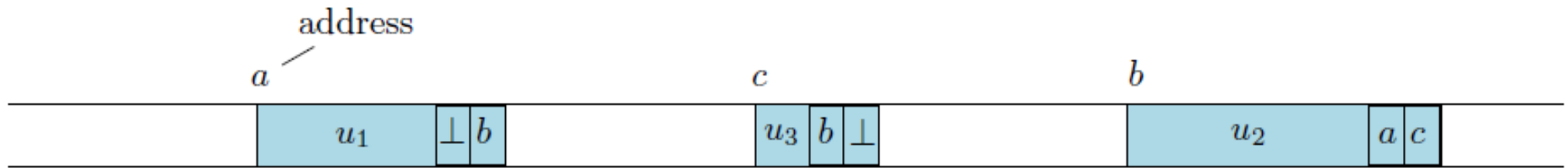
Array Implementation Efficiency

- ❑ Accessing an element
 - $O(1)$
- ❑ Iterating over elements
 - $O(n)$
- ❑ Insert / Delete element
 - $O(n)$ // why?
- ❑ Memory usage
 - $O(n)$

Linked Implementation

- ❑ Sequence of nodes
- ❑ Node data is stored in consecutive memory cells
- ❑ Every node stores a reference to its following node
 - possibly previous as well
- ❑ Head – first node
- ❑ Tail – last node

Linked Implementation



- u_1, u_2, u_3 – Three nodes in a linked list
- a, b, c – Addresses in memory
- $\perp$ – Represents null
- Nodes can be stored anywhere in memory
 - addresses of the nodes do not need to follow the ordering of the nodes in the linked list

Linked Implementation

- Size can grow and shrink as needed
 - Essentially limited by available memory
 - ◆ constrained only by needing consecutive memory cells of the size of a single node

Linked Implementation Efficiency

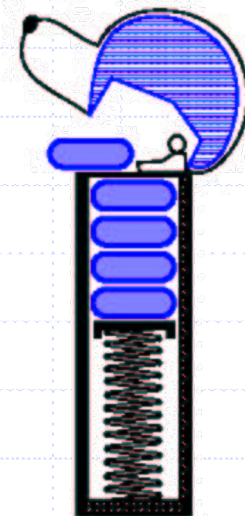
- Accessing an element
 - $O(1)$
- Iterating over elements
 - $O(n)$
- Insert / Delete element at known position
 - $O(1)$
- Memory usage
 - $O(n)$

Abstract Data Types (ADTs)

- Abstraction of a data structure
- Specifies:
 - Data stored
 - Operations on the data
 - Error conditions associated with operations
- Example: ADT modelling a simple stock trading system
 - Data stored are buy/sell orders
 - Operations supported are
 - ◆ order **buy**(stock, shares, price)
 - ◆ order **sell**(stock, shares, price)
 - ◆ void **cancel**(order)
 - Error conditions
 - ◆ buy/sell a nonexistent stock
 - ◆ cancel a nonexistent order

Stack ADT

- ❑ Stores arbitrary objects
- ❑ Insertions and deletions are last-in, first-out (LIFO)
- ❑ Think of a spring-loaded plate dispenser
 - insertions at top of stack
 - removals from top of stack



Stack Operations

- `push(T)` – inserts an element
- `T pop()` – removes and returns the last inserted element
- `T top()` – returns the last inserted element without removing it
- `integer size()` – returns the number of elements stored
- `boolean isEmpty()` – indicates whether no elements are stored

Stack Interface in Java

- Assumes `null` is returned from `top()` and `pop()` when stack is empty
- Different from the built-in Java class `java.util.Stack`

```
public interface Stack<T>
{
    int size();
    boolean isEmpty();
    T top();
    void push(T element);
    T pop();
}
```

Example

Method	Return Value	Stack Contents
push(5)	—	(5)
push(3)	—	(5, 3)
size()	2	(5, 3)
pop()	3	(5)
isEmpty()	false	(5)
pop()	5	()
isEmpty()	true	()
pop()	null	()
push(7)	—	(7)
push(9)	—	(7, 9)
top()	9	(7, 9)
push(4)	—	(7, 9, 4)
size()	3	(7, 9, 4)
pop()	4	(7, 9)
push(6)	—	(7, 9, 6)
push(8)	—	(7, 9, 6, 8)
pop()	8	(7, 9, 6)

Exceptions vs. Returning Null

- Attempting to execute an ADT operation may cause an error condition
 - pop an empty stack
- Need to decide what to do and how to report the error
 - raise an exception
 - return a special value
- Previous Stack ADT returns a special value
 - `null`
- Allow **pop** and **top** to be performed even if the stack is empty
 - client code needs to check for `null` return values
 - versus catching an exception

Stack Interface with Exceptions

```
public interface Stack<T> {  
    int size();  
    boolean isEmpty();  
    T top() throws NoSuchElementException;  
    void push(T element);  
    T pop() throws NoSuchElementException;  
}
```

□ What about **push**?

Applications of Stacks

- Direct applications
 - Page-visited history in a Web browser
 - Undo sequence of edits in a text editor
 - Chain of method calls in the Java Virtual Machine (the “call stack”)
- Indirect applications
 - Auxiliary data structure for algorithms
 - Component of other data structures

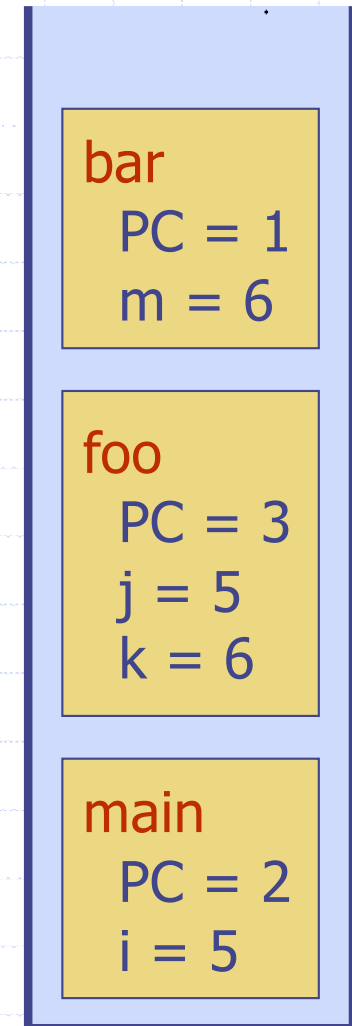
Method Stack in JVM

- ❑ Java Virtual Machine (JVM) keeps track of the chain of active methods with a stack
- ❑ When a method is called, the JVM pushes a frame containing
 - local variables and return value
 - program counter, keeping track of the statement being executed
- ❑ When a method ends, its frame is popped from the stack and control is passed to the method on top of the stack
- ❑ Allows for recursion

```
main() {  
    int i = 5;  
    foo(i);  
}
```

```
foo(int j) {  
    int k;  
    k = j+1;  
    bar(k);  
}
```

```
bar(int m) {  
    ...  
}
```



Array-based Stack

- ❑ Simple Stack implementation
- ❑ Add elements from left to right
- ❑ Variable keeps track of index of top element

Algorithm *size()*

return $t + 1$

Algorithm *pop()*

if *isEmpty()* **then**

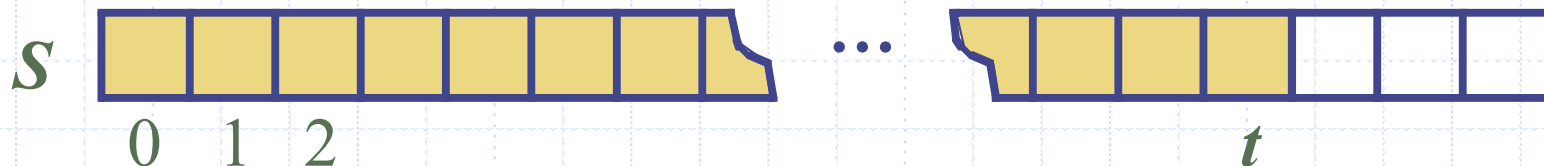
throw

NoSuchElementException

else

$t \leftarrow t - 1$

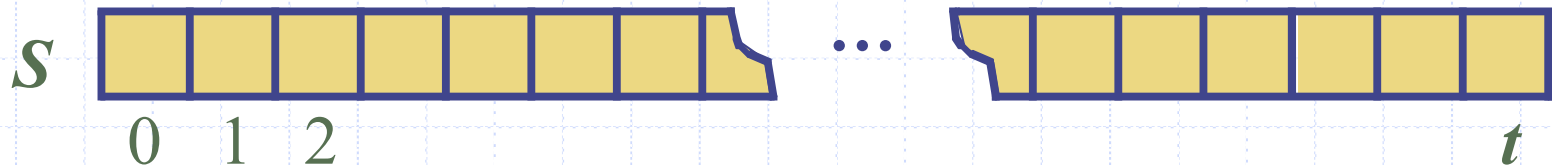
return $S[t + 1]$



Array-based Stack (cont.)

- ❑ Array storing the stack may become full
- ❑ Push operation throws `IllegalStateException`
 - limitation of array implementation
 - not intrinsic to the Stack ADT
 - could specialise to be `StackOverflowException`

```
Algorithm push(item)
  if  $t = S.length - 1$  then
    throw
      IllegalStateException
  else
     $t \leftarrow t + 1$ 
     $S[t] \leftarrow item$ 
```



Performance and Limitations

□ Performance

- Let n be the number of elements in the stack, and let $N \geq n$ be the fixed size of the array
- Space used is $O(N)$
- Each operation runs in time $O(1)$

□ Limitations

- Maximum size of the stack must be defined a priori and cannot be changed
- Pushing a new element onto a full stack causes an implementation-specific exception

Array-based Stack in Java

```
public class ArrayStack<T>
    implements Stack<T> {

    // Holds the stack elements
    private T[] stack;

    // Index to top element
    private int top = -1;

    public ArrayStack(int capacity) {
        stack =
            (T[]) new Object[capacity];
    }
}
```

```
public T pop() {
    if (isEmpty())
        throw new
            NoSuchElementException();
    T temp = stack[top];
    // Facilitate garbage collection:
    stack[top] = null;
    top = top - 1;
    return temp;
}

// ... (other methods of Stack interface)
```

Java Example: Reverse an Array

```
public class Tester<V> {  
    // ... other methods  
    public void reverseArray(V a[]) {  
        Stack<V> s = new ArrayStack<>(a.length);  
        for(V value: a) {  
            s.push(value);  
        }  
        int i = 0;  
        while(!s.isEmpty()) {  
            a[i++] = s.pop();  
        }  
    }  
}
```

Parentheses Matching

- Each “(”, “{”, or “[” must be paired with a matching “)”, “}”, or “]”
 - correct: ()(()){([())}
 - correct: ((())(()){([())}
 - incorrect:)(()){([())}
 - incorrect: ({ []})
 - incorrect: (

Parenthesis Matching (Java)

```
public static boolean isMatched(String expression) {  
    final String opening = "{[("; // opening delimiters  
    final String closing = ")}]"; // respective closing delimiters  
    Stack<Character> buffer = new LinkedStack<>( );  
    for (char c : expression.toCharArray( )) {  
        if (opening.indexOf(c) != -1) // this is a left delimiter  
            buffer.push(c);  
        else if (closing.indexOf(c) != -1) { // this is a right delimiter  
            if (buffer.isEmpty( )) // nothing to match with  
                return false;  
            if (closing.indexOf(c) != opening.indexOf(buffer.pop( )))  
                return false; // mismatched delimiter  
        }  
    }  
    return buffer.isEmpty( ); // here if all opening delimiters matched  
}
```

HTML Tag Matching

- ❑ Correct XHTML: each `<name>` should pair with a matching `</name>`

```
<body>
<center>
<h1> The Little Boat </h1>
</center>
<p> The storm tossed the little
boat like a cheap sneaker in an
old washing machine. The three
drunken fishermen were used to
such treatment, of course, but
not the tree salesman, who even as
a stowaway now felt that he
had overpaid for the voyage. </p>
<ol>
<li> Will the salesman die? </li>
<li> What color is the boat? </li>
<li> And what about Naomi? </li>
</ol>
</body>
```

The Little Boat

The storm tossed the little boat like a cheap sneaker in an old washing machine. The three drunken fishermen were used to such treatment, of course, but not the tree salesman, who even as a stowaway now felt that he had overpaid for the voyage.

1. Will the salesman die?
2. What color is the boat?
3. And what about Naomi?

HTML Tag Matching (Java)

```
public static boolean isHTMLMatched(String html) {  
    Stack<String> buffer = new LinkedStack<>( );  
    int j = html.indexOf('<'); // find first '<' character (if any)  
    while (j != -1) {  
        int k = html.indexOf('>', j+1); // find next '>' character  
        if (k == -1)  
            return false; // invalid tag  
        String tag = html.substring(j+1, k); // strip away < >  
        if (!tag.startsWith("/")) // this is an opening HTML tag  
            buffer.push(tag); // note: only storing tags on the stack  
        else { // this is a closing tag  
            if (buffer.isEmpty( ))  
                return false; // no tag to match  
            if (!tag.substring(1).equals(buffer.pop( ))) // skip over '/' of tag  
                return false; // mismatched tag  
        }  
        j = html.indexOf('<', k+1); // find next '<' character (if any)  
    }  
    return buffer.isEmpty( ); // were all opening tags matched?  
}
```

Evaluating Arithmetic Expressions

Slide by Matt Stallmann
included with permission.

$$14 - 3 \times 2 + 7 = (14 - (3 \times 2)) + 7$$

Operator precedence

- $\times$ has precedence over $+$ and $-$

Associativity

- operators of the same precedence group are evaluated from left to right
- Example: $(x - y) + z$ rather than $x - (y + z)$

Idea: push each operator on the stack, but first pop and perform higher and *equal* precedence operations.

Algorithm for Evaluating Expressions

Slide by Matt Stallmann
included with permission.

Two Stacks

- opStk holds operators
- valStk holds values
- Use \$ as special “end of input” token with lowest precedence

Algorithm doOp():

```
x ← valStk.pop()
y ← valStk.pop()
op ← opStk.pop()
valStk.push( y op x )
```

Algorithm repeatOps(refOp):

```
while ( valStk.size() > 1
      ^ prec(refOp)
      ≤ prec(opStk.top() )
    doOp()
```

Algorithm EvalExp():

Input: stream of tokens
representing an arithmetic
expression

Output: value of the expression

while there's another token z

if isNumber(z) **then**

 valStk.push(z)

else

 repeatOps(z)

 opStk.push(z)

repeatOps(\$);

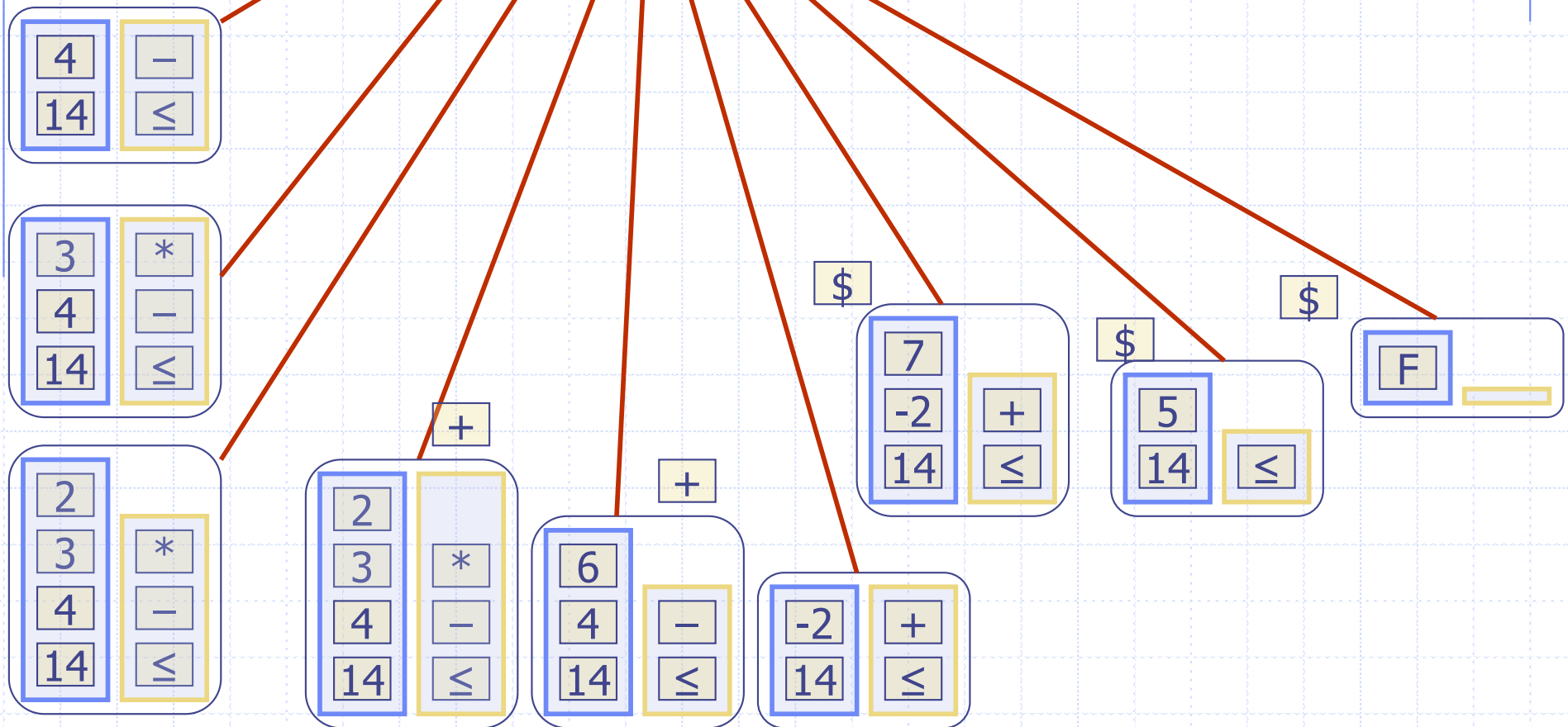
return valStk.top()

Algorithm on an Example Expression

Slide by Matt Stallmann included with permission.

$14 \leq 4 - 3 \times 2 + 7$

Operator $\leq$ has lower precedence than $+$ or $-$



Reverse Polish Notation

ShuntingYardEvaluator.java

Enter an infix expression:

$((3+4)*(5-2))$

Converting to Postfix:

Action	Stack	Output
Processed (	[(]	[]
Processed (	[(, (]	[]
Processed +	[(, (, +]	[3]
Processed)	[(]	[3, 4, +]
Processed *	[(, *]	[3, 4, +]
Processed (	[(, *, (]	[3, 4, +]
Processed -	[(, *, (, -]	[3, 4, +, 5]
Processed)	[(, *]	[3, 4, +, 5, 2, -]
Processed)	[]	[3, 4, +, 5, 2, -, *]

Postfix expression: 3 4 + 5 2 - *

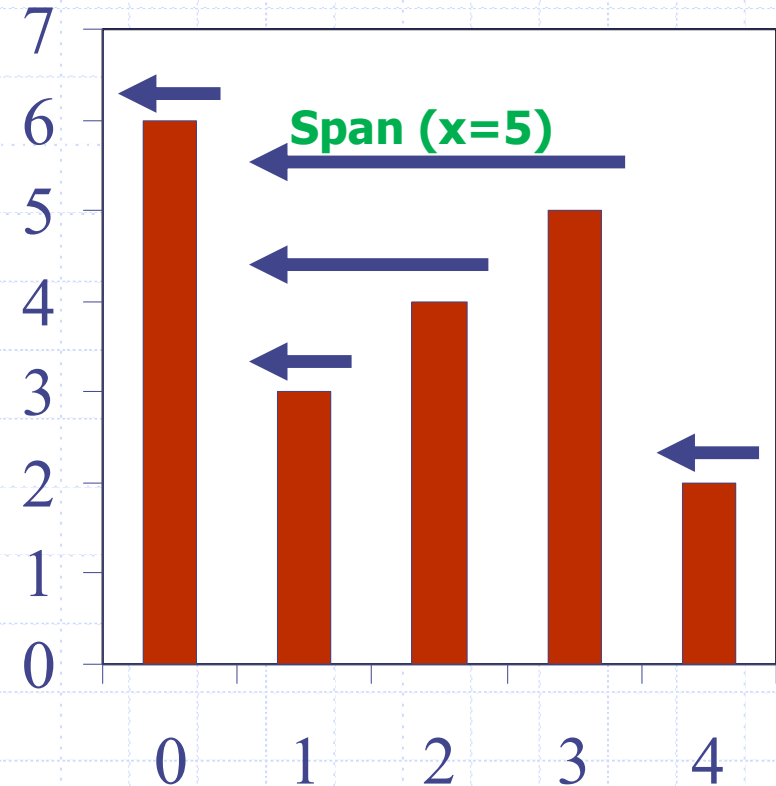
Evaluating Postfix:

Action	Stack
Push 3	[3]
Push 4	[3, 4]
Apply +	[7]
Push 5	[7, 5]
Push 2	[7, 5, 2]
Apply -	[7, 3]
Apply *	[21]

Result of evaluation: 21

Computing Spans

- Given an array X , the **span** $S[i]$ of $X[i]$ is the maximum number of consecutive elements $X[j]$ immediately preceding $X[i]$, such that $X[j] \leq X[i]$
 - span includes $X[i]$
- Spans have applications to financial analysis
 - e.g. stock at 52-week high



X	6	3	4	5	2
S	1	1	2	3	1

Quadratic Algorithm

Algorithm *spans1*(X, n)

Input array X of n integers

Output array S of spans of X

$S \leftarrow$ new array of n integers

for $i \leftarrow 0$ **to** $n - 1$ **do**

$s \leftarrow 1$

while $s \leq i \wedge X[i - s] \leq X[i]$

$s \leftarrow s + 1$

$S[i] \leftarrow s$

return S

operations

n

n

n

$1 + 2 + \dots + (n - 1)$

$1 + 2 + \dots + (n - 1)$

n

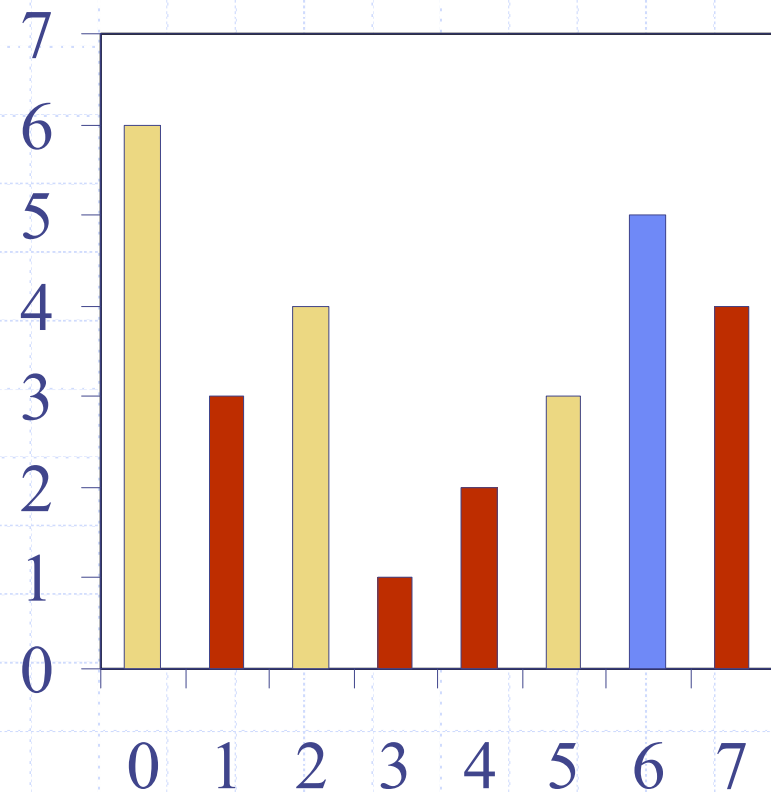
1

◆ $f(n) = n^2 + n$

◆ Runs in $O(n^2)$ time

Computing Spans with a Stack

- Stack holds indices of elements visible when “looking back”
- Scan the array from left to right
 - Let i be the current index
 - Pop indices from stack until we find index j such that $X[j] > X[i]$
 - Set $S[i] \leftarrow i - j$
 - Push i onto stack



Linear Time Algorithm

- ❑ Uses a stack as an auxiliary data structure
- ❑ Each index of the array
 - ❑ pushed onto the stack exactly once !!
 - ❑ popped from the stack at most once !!
- ❑ Statements in the while-loop are executed at most n times
- ❑ Runs in $O(n)$ time

Algorithm <i>spans2</i> (X, n)	#
$S \leftarrow$ new array of n integers	n
$A \leftarrow$ new empty stack	1
for $i \leftarrow 0$ to $n - 1$ do	n
while $(\neg A.isEmpty() \wedge$	
$X[A.top()] \leq X[i])$ do	n
$A.pop()$	n
if $A.isEmpty()$ then	n
$S[i] \leftarrow i + 1$	n
else	
$S[i] \leftarrow i - A.top()$	n
$A.push(i)$	n
return S	1

```

public static int[] spans2(int[] X) {
    int    n = X.length;
    int[]  S = new int[n];
    Stack<Integer> A = new ArrayStack<>(n);

```

n=5

	0	1	2	3	4
X	6	3	4	5	2
S					

```

    for (int i = 0; i < n; i++) {
        while (!A.isEmpty() && (X[A.top()] <= X[i])) {
            A.pop(); // note: the only statement in the loop !!
        }
        if (A.isEmpty()) {
            S[i] = i+1;
        } else {
            S[i] = i - A.top();
        }
        A.push(i);
    }
    return S;
}

```

A

i=0

```

public static int[] spans2(int[] X) {
    int    n = X.length;
    int[]  S = new int[n];
    Stack<Integer> A = new ArrayStack<>(n);

```

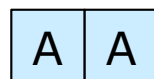
n=5

	0	1	2	3	4
X	6	3	4	5	2
S	1				

```

    for (int i = 0; i < n; i++) {
        while (!A.isEmpty() && (X[A.top()] <= X[i])) {
            A.pop();
        }
        if (A.isEmpty()) {
            S[i] = i+1; // s[0] = 1
        } else {
            S[i] = i - A.top();
        }
        A.push(i);
    }
    return S;
}

```



i=0

```

public static int[] spans2(int[] X) {
    int    n = X.length;
    int[]  S = new int[n];
    Stack<Integer> A = new ArrayStack<>(n);

```

n=5

	0	1	2	3	4
X	6	3	4	5	2
S	1				

```

    for (int i = 0; i < n; i++) {
        while (!A.isEmpty() && (X[A.top()] <= X[i])) {
            A.pop();
        }
        if (A.isEmpty()) {
            S[i] = i+1;
        } else {
            S[i] = i - A.top();
        }
        A.push(i);
    }
    return S;
}

```



		0
A	A	A

i=0

```

public static int[] spans2(int[] X) {
    int    n = X.length;
    int[]   S = new int[n];
    Stack<Integer> A = new ArrayStack<>(n);

```

n=5

	0	1	2	3	4
X	6	3	4	5	2
S	1	1			

```

    for (int i = 0; i < n; i++) {
        while (!A.isEmpty() && (X[A.top()] <= X[i])) {
            A.pop();
            // (X[0] = 6) <= (X(1) = 3)
        }
        if (A.isEmpty()) {
            S[i] = i+1;
        } else {
            S[i] = i - A.top(); // S[1] = 1 - 0 = 1
        }
        A.push(i);
    }

```

```

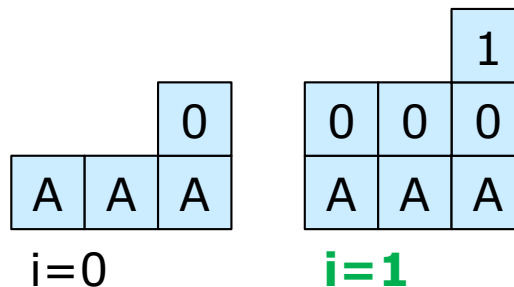
    return S;

```

```

}

```



```

public static int[] spans2(int[] X) {
    int    n = X.length;
    int[]  S = new int[n];
    Stack<Integer> A = new ArrayStack<>(n);

```

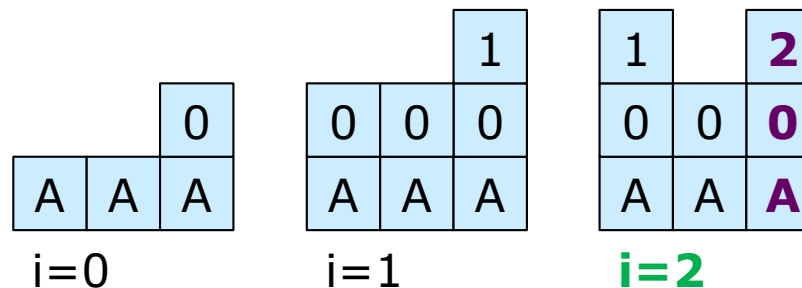
n=5

	0	1	2	3	4
X	6	3	4	5	2
S	1	1	2		

```

    for (int i = 0; i < n; i++) {
        while (!A.isEmpty() && (X[A.top()] <= X[i])) {
            A.pop();
            // (X[1] = 3) <= (X[2] = 4)
        }
        if (A.isEmpty()) {
            S[i] = i+1;
        } else {
            S[i] = i - A.top(); // S[2] = 2 - 0
        }
        A.push(i);
    }
    return S;
}

```



```

public static int[] spans2(int[] X) {
    int    n = X.length;
    int[]  S = new int[n];
    Stack<Integer> A = new ArrayStack<>(n);

```

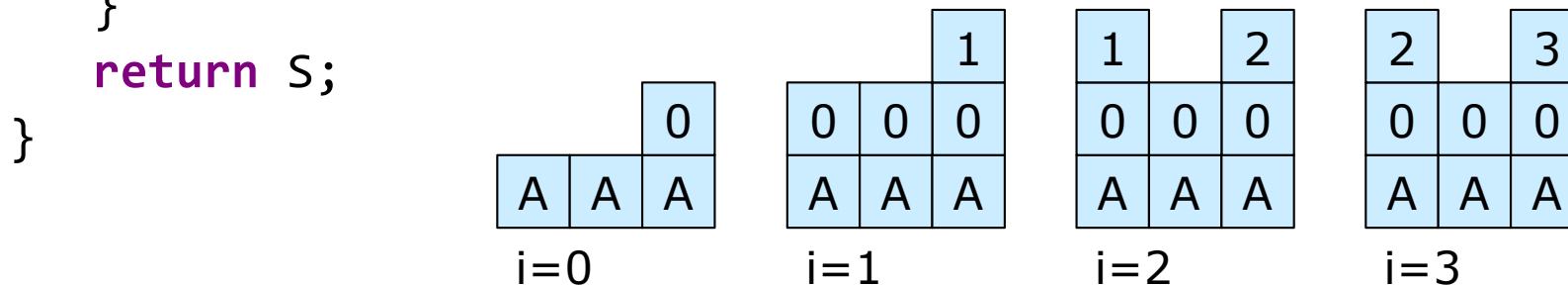
n=5

	0	1	2	3	4
X	6	3	4	5	2
S	1	1	2	3	

```

    for (int i = 0; i < n; i++) {
        while (!A.isEmpty() && (X[A.top()] <= X[i])) {
            A.pop();
        }
        if (A.isEmpty()) {
            S[i] = i+1;
        } else {
            S[i] = i - A.top(); // S[3] = 3 - 0
        }
        A.push(i);
    }
    return S;

```



```

public static int[] spans2(int[] X) {
    int    n = X.length;
    int[]  S = new int[n];
    Stack<Integer> A = new ArrayStack<>(n);

```

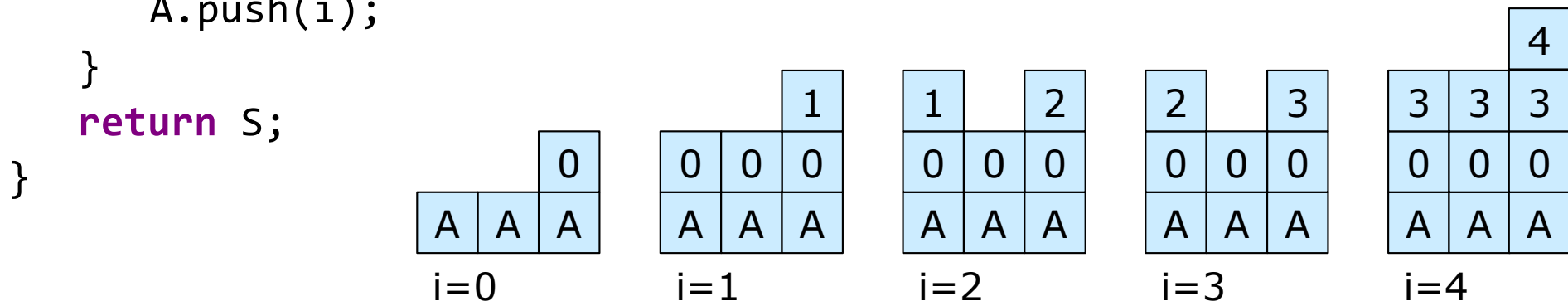
n=5

	0	1	2	3	4
X	6	3	4	5	2
S	1	1	2	3	1

```

    for (int i = 0; i < n; i++) {
        while (!A.isEmpty() && (X[A.top()] <= X[i])) {
            A.pop();
        }
        if (A.isEmpty()) {
            S[i] = i+1;
        } else {
            S[i] = i - A.top(); // s[4] = 4 - 3 = 1
        }
        A.push(i);
    }
    return S;

```



Summary

- ❑ Implementing linear structures
 - array
 - linked list
- ❑ Stack ADT
- ❑ Stack applications